INTERIM REPORT

AI-Powered Comparison of IT Programs Across Ontario Colleges

Course: Capstone Project

Group: 3

Date: March 8, 2026

Team Members

Amit Sharma

Ahemadashraf Pathan

Sahil Varudi

Sahil Jajadiya

Deepesh Dixit

1. Executive Summary

This is the interim report for our Capstone project, "AI-Powered Comparison of IT Programs Across Ontario Colleges." In short — we're building a web platform where students, advisors, recruiters, and policy people can look up, compare, and get recommendations for IT programs at Ontario colleges.

We've made a lot of progress. The dataset is collected and cleaned (3,273 records across 26 colleges), we've done a full EDA, the backend API is up and running with eight endpoints, and the entire frontend is functional. Right now the recommendation engine is rule-based — it scores programs based on keyword matching against a student's input — which honestly works better than we expected as a first version.

What's still left is the machine learning side: replacing the keyword scorer with trained models (KMeans clustering and a Random Forest or XGBoost classifier), adding proper NLP-based skill extraction, writing tests, and finishing all the documentation before the final submission.

2. Introduction and Background

When we first sat down to pick a project, one thing that frustrated all of us was how hard it is to actually compare IT programs across Ontario colleges. Each college has its own website, its own way of describing programs, and there's no single place to see them side by side. A student who wants to know 'what's the difference between Seneca's cybersecurity diploma and George Brown's?' has to open five tabs and still won't get a clean answer.

That's the problem we're trying to fix. Ontario has 24 publicly funded colleges, and between them they offer hundreds of technology-related programs — diplomas, certificates, graduate certificates, even bachelor's degrees. The programs differ in length, credential level, delivery format, the specific skills they focus on, and where they're offered. For a student fresh out of high school or looking to upskill, navigating that is genuinely confusing.

The same problem shows up in a different form for academic advisors. They counsel students every day but don't always have a straightforward way to see how programs are clustered by skill set. Recruiters have their own version: they want to know which colleges are producing graduates with specific technical skills, and that information just isn't compiled anywhere.

Our project tries to solve all of this in one place. The idea is a platform that doesn't just list programs but actually understands a user's profile and helps them make a decision — whether that user is a student, an advisor, a recruiter, or a policy analyst tracking regional skill supply.

3. Project Objectives

When we scoped the project in Milestone 1, we landed on seven main things we wanted the platform to do:

- Build out a proper dataset of Ontario IT programs — not just names and colleges, but credential type, duration, delivery format, region, and especially program descriptions.

- Build a recommendation engine that takes a student's background and goals and returns a ranked list of programs that actually fit them.
- Create an advisor-facing view where programs are grouped by skill cluster, so advisors can quickly see what's available in, say, cybersecurity or data analytics.
- Let colleges benchmark themselves against each other — comparing program counts, credential types, and skill coverage.
- Give recruiters a way to search by skill keyword and see which colleges have the most matching programs.
- Show a policy-level view of how IT program supply is distributed across regions, useful for spotting gaps in Ontario's tech education landscape.
- Package all of this in a browser-based web app that's easy to use — not just a proof-of-concept notebook.

4. Work Completed to Date

4.1 Data Collection and Preparation

Everything in this project starts with the data, so this was the first thing we tackled. We pulled program information from Ontario college websites and the Ontario Colleges central portal. Getting it all into a consistent shape took more effort than expected — the raw data had a lot of inconsistencies that needed sorting out (more on that in the Challenges section).

After combining the sources, we deduplicated on a composite key: college + campus + program name + program code + intake term + start date. Here's what the final dataset looks like:

Attribute	Value
Total programs (after dedup)	2039
Colleges covered	26
Credential types	8
Columns per record	14

The 14 fields per program: college_name, campus_name, program_name, credential_type, credential_group, duration_months, duration_years, program_category, delivery_format, source_url, details_url, program_level, program_length, and program_description. That last field is the one we're most interested in going forward — it's the raw material for the NLP work planned next.

4.2 Exploratory Data Analysis

Once the data was clean, we ran a full EDA pass in a Jupyter notebook. A few things stood out:

- Diplomas and Graduate Certificates dominate. Bachelor's degrees exist but are relatively uncommon.
- Most programs are full-time and in-person, but there's a decent chunk that's online or hybrid — more than we expected.

- Duration varies a lot — from under a year for short certificates to four years for bachelor's programs. Median is around two years.
- Program descriptions are inconsistent. Some colleges write detailed learning outcomes; others just list course names. That variability will need preprocessing.
- Colleges in the GTA offer noticeably more IT programs than smaller-city colleges. That geographic concentration is pretty clear in the numbers.
- Many rows had 'Not Available' in the duration columns, which required a handling decision on our end.

4.3 System Architecture Design

We spent time early on thinking about how to structure things so different pieces could be built in parallel. We landed on four layers:

- Data Ingestion — scraping, validation, and cleaning into structured CSV/Parquet format.
- Feature Engineering — skill extraction from program descriptions using NLP (spaCy + TF-IDF), generating feature vectors.
- ML Modeling — KMeans clustering for grouping programs, Random Forest / XGBoost for fit scoring, and a cosine similarity recommender that ties it together.
- Application — FastAPI backend with a REST API, browser-based frontend on top.

The layering was intentional. It meant the data team could work without blocking the backend developer, and the frontend could be built against a working API before the ML models were ready.

4.4 Backend API Development

The backend is a FastAPI app in app.py. It reads the CSV at startup, filters to IT programs, and exposes these eight endpoints:

Endpoint	Method	What it does
/api/health	GET	Confirms the server is running
/api/colleges	GET	List of all colleges in the dataset
/api/programs	GET	Searchable, filterable program list
/api/recommendations	POST	Takes a student profile, returns ranked programs
/api/skill-clusters	GET	Programs grouped by skill area
/api/stats	GET	Counts, breakdowns, summary numbers
/api/benchmark	GET	College-level comparison data
/api/policy	GET	Regional program supply breakdown

4.5 Frontend Web Application

The frontend is a single HTML page — no framework, just HTML, CSS, and JavaScript. We kept it simple on purpose. The JS fetches from the backend and updates the page dynamically. What's working:

- Recommendations form — enter background and goals, get back a ranked list of programs as cards.

- Skill clusters view — programs by track (Cybersecurity, Data & AI, Software Dev, Cloud & DevOps, Networks, IT Support), useful for advisors.
- Benchmarking table — colleges compared side by side on program counts, credential mix, and skill coverage.
- Data explorer — sortable, filterable table of the full dataset.
- Olivia chatbot — a rule-based assistant that helps users navigate the app using live data from the API.

5. Technical Implementation Details

5.1 Technology Stack

Layer	Technology
Backend	Python 3.x, FastAPI, Pydantic
Data handling	Pandas, Python's built-in CSV module
NLP (right now)	Regex and keyword matching
NLP (next step)	spaCy, TF-IDF via scikit-learn
Frontend	Plain HTML5, CSS3, vanilla JavaScript
Charts	Chart.js
Version control	Git / GitHub
Project tracking	Jira

5.2 Dataset Loading

The CSV sits at the project root. The backend reads it once at startup, runs the IT keyword filter, and holds the filtered records in memory for the life of the process. This keeps response times consistent — there's no database hit per request.

5.3 Scoring Formula

The current scoring for each program:

Score = (skills_match x 3) + (goals_match x 2) + (education_match x 1)
+ delivery_bonus + region_bonus + duration_bonus

Programs scoring zero are excluded entirely, so every result has at least some relevance to the input.

5.4 IT Keyword Filter

We filter to IT programs using 17 domain keywords checked against program names: "computer," "software," "network," "data," "cyber," "cloud," "programming," "web," "systems," and similar. The filter runs at load time so the API only ever works with the relevant subset.

6. Challenges and How We Addressed Them

Challenge 1: Colleges Don't Use the Same Vocabulary

Problem: This sounds minor, but it caused real headaches early on. One college writes "Full Time," another writes "Full-Time," a third writes "FT." Same issue with credential names, campus names, and program categories — every source had its own conventions.

How we fixed it: We fixed it with a normalization function that runs on every string before comparison: lowercase, strip punctuation, collapse whitespace. Not glamorous, but it made matching work reliably across all 26 colleges.

Challenge 2: Duration Data Was Unreliable

Problem: A lot of records had "Not Available" in the duration_months and duration_years columns. We couldn't drop those rows — some were otherwise good programs. And we couldn't fill in the durations without fabricating data.

How we fixed it: The solution was a neutral score on the duration component rather than a penalty. Those programs still show up in results if they match well on skills and goals — they just don't get the duration bonus.

Challenge 3: Separating IT Programs from Everything Else

Problem: The raw dataset had programs from every faculty. Defining 'IT' cleanly is harder than it sounds — 'Computer Applications' is obvious, but 'Health Informatics' or 'Digital Marketing' are borderline.

How we fixed it: We iterated a few times on the keyword list and ended up with 17 terms. We validated the filter by manually checking a sample of what it included and excluded.

Challenge 4: Frontend Without a Build System

Problem: We chose not to use React or any framework. That paid off in simplicity but meant app.js grew pretty long as features were added.

How we fixed it: We kept it manageable with section comment headers per feature area and a consistent fetch-then-render pattern throughout. The backend serving static files directly also kept deployment simple.

Challenge 5: Coordinating Five People Across Multiple Components

Problem: Early on there were moments where API changes broke something on the frontend without anyone realizing until later. Five people across data, backend, frontend, and docs needs more coordination than it might seem.

How we fixed it: We got better at this by being explicit about task ownership in Jira and communicating before merging anything that touched the API contract. Short check-ins at the start of each session helped a lot.

7. Work Remaining

7.1 Machine Learning

The main remaining task is replacing the scoring engine with trained models:

- KMeans clustering on program feature vectors, to replace the six hand-defined skill clusters we have now. The goal is to let the data decide how programs group naturally.
- Random Forest or XGBoost classifier trained on student profile and program attribute pairs to predict fit score. We'll use synthetic or manually labeled profiles since we don't have real interaction history.
- Sentence-BERT embeddings on program descriptions, if there's time — to test whether semantic similarity outperforms TF-IDF for the matching step.

7.2 NLP Skill Extraction

Right now skill matching is keyword overlap. What we want is spaCy pulling a structured skill list out of each program description — 'Python,' 'AWS,' 'network administration,' 'machine learning' — so the recommender can do a real skill-to-skill comparison.

7.3 Model Evaluation

Once models exist, we need actual numbers:

- Clustering: silhouette score and a manual review of whether the clusters make sense.
- Classifier: accuracy, precision, recall, F1, AUC on a held-out test set.
- Recommendations: NDCG and hit rate at K using hand-crafted gold-standard student profiles.

7.4 Testing

- Unit tests per API endpoint.
- End-to-end tests for the full recommendation flow.
- Edge case coverage: empty input, unknown region, extreme duration values.

7.5 Final Documentation

- Swagger schemas need proper descriptions (the page auto-generates but is sparse right now).
- Updated architecture diagram.
- User guide per persona (student, advisor, recruiter, policy analyst).
- Final project report.

8. Timeline and Milestones

Milestone	Status	Notes
Milestone 1 – Project Proposal	Completed	Scope agreed and approved
Milestone 2 – Epic Definition, Technical Planning	Completed	Architecture done, dataset collected
Data Collection and Cleaning	Completed	3,273 records, 26 colleges
Exploratory Data Analysis	Completed	Notebook finished
Backend API (rule-based)	Completed	All 8 endpoints working
Frontend Web Application	Completed	All sections live
ML Model Development	In Progress	KMeans and classifier being built
NLP Skill Extraction	Not Started	Next sprint
Model Evaluation	Not Started	After model training
Testing	Not Started	Penultimate sprint
Final Report and Presentation	Not Started	End of semester

9. Individual Contributions

Team Member	Primary Responsibilities
Amit Sharma	Data scraping, cleaning, ML research
Ahemadashraf Pathan	Backend API, recommendation engine, overall architecture, Frontend structure and styling — index.html, styles.css,
Sahil Varudi	Meeting Minutes, Interim Report Along with Deepesh
Sahil Jajadiya	EDA notebook, Jira board management
Deepesh Dixit	Documentation, Jira board management

All five of us reviewed and tested each other's work throughout the sprints, so the lines aren't perfectly clean — but the table above captures who led each area.

10. Conclusion

Honestly, we're in a better place than we expected at this point. Coming into the project, building a full recommendation platform felt ambitious. What we have now — a working web app, a clean dataset, and a functional API — is a solid base to build the ML layer on.

That said, the next phase is the harder one. Rule-based scoring is forgiving to build; a properly trained and evaluated ML pipeline is not. We're being realistic about the time we have and have the remaining work tracked in Jira. If something has to be cut, semantic embeddings are the first thing to deprioritize.

The goal for the final submission is a platform that works well enough for someone to actually use it to make a real program decision. That's the bar we've set.